Міністерство освіти і науки України

Національний технічний університет України

Київський політехнічний інститут імені Ігоря Сікорського

Лабораторна робота №6

«Міграції схем за допомогою Prisma ORM»

Роботу виконали
Студенти групи ІО-45, ІО-43
Ренцевич Д. , Гапон А.
Роботу перевірив:
Русінов В.В.

Київ 2026

ЛАБОРАТОРНА РОБОТА № 6

Тема: Міграції схем за допомогою Prisma ORM.

Мета: Розвинути схему бази даних за допомогою інструментів міграції Prisma ORM, імітуючи зміни в реальній базі даних. Навчитися безпечно оновлювати виробничі бази даних з часом, використовуючи поступові міграції.

Завдання:

1.Використати Prisma ORM для підключення до існуючої БД та зчитування її поточної схеми.

2.Змоделювати зміни у файлі schema.prisma (додати таблицю, змінити існуючу таблицю, видалити стовпець).

3.Згенерувати та застосувати міграції SQL за допомогою команди prisma migrate.

4.Виконати базові запити через клієнт Prisma для перевірки успішності внесених змін.

Зміна схеми бази даних (schema.prisma):

Відповідно до завдання, у файл schema.prisma було внесено три основні зміни до початкової структури бази даних готелю:

Створено нову таблицю review для збереження відгуків гостей (поля: id, рейтинг, коментар, зв'язки з гостем та кімнатою).

Додано нове поле is_vip (логічного типу Boolean) до існуючої таблиці guest для ідентифікації VIP-клієнтів.

Видалено поле description з таблиці room_type для оптимізації структури.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// 1. З таблиці room_type ми ВИДАЛИЛИ поле description
model room_type {
  type_id    Int       @id @default(autoincrement())
  type_name  String    @db.VarChar(50)
  base_price Decimal   @db.Decimal(8, 2)
  room       room[]
}

// 2. У таблицю guest ми ДОДАЛИ поле is_vip
model guest {
  guest_id         Int       @id @default(autoincrement())
  name             String    @db.VarChar(50)
  surname          String    @db.VarChar(50)
  passport_details String    @unique @db.VarChar(50)
  phone            String?   @db.VarChar(20)
  email            String?   @unique @db.VarChar(100)
  is_vip           Boolean   @default(false) // Ось це нове поле
  booking          booking[]
  review           review[]  // Зв'язок з новою таблицею відгуків
}

model room {
  room_id     Int       @id @default(autoincrement())
  room_number String    @unique @db.VarChar(10)
  status      String?   @default("вільний") @db.VarChar(20)
  type_id     Int?
  room_type   room_type? @relation(fields: [type_id], references:
[type_id], onDelete: Restrict)
  booking     booking[]
  review      review[]   // Зв'язок з новою таблицею відгуків
}

model booking {
  booking_id     Int       @id @default(autoincrement())
  check_in_date  DateTime  @db.Date
  check_out_date DateTime  @db.Date
  booking_status String?   @default("підтверджено") @db.VarChar(20)
  guest_id       Int?
  room_id        Int?
```

```
  guest          guest?    @relation(fields: [guest_id], references:
[guest_id], onDelete: Cascade)
  room           room?     @relation(fields: [room_id], references:
[room_id], onDelete: Restrict)
  payment        payment[]
}

model payment {
  payment_id   Int       @id @default(autoincrement())
  amount       Decimal   @db.Decimal(8, 2)
  payment_date DateTime? @default(now()) @db.Timestamp(6)
  booking_id   Int?
  booking      booking?  @relation(fields: [booking_id], references:
[booking_id], onDelete: Cascade)
}

// 3. Це абсолютно НОВА ТАБЛИЦЯ для відгуків
model review {
  review_id Int     @id @default(autoincrement())
  rating    Int
  comment   String?
  guest_id  Int
  room_id   Int
  guest     guest   @relation(fields: [guest_id], references:
[guest_id], onDelete: Cascade)
  room      room    @relation(fields: [room_id], references:
[room_id], onDelete: Cascade)
}
```

Застосування міграцій:

Для того, щоб перенести зміни з файлу schema.prisma у реальну базу даних PostgreSQL, було використано команду генерації міграцій:

```
npx prisma migrate dev --name final-migration
```

```
C:\Users\user\Desktop\Lab 6>npx prisma migrate dev --name final-migration
Environment variables loaded from .env
Prisma schema loaded from prisma\schema.prisma
Datasource "db": PostgreSQL database "postgres", schema "public" at "localhost:5432"

Drift detected: Your database schema is not in sync with your migration history.

The following is a summary of the differences between the expected database schema given

It should be understood as the set of changes to get from the expected schema to the actu

If you are running this the first time on an existing database, please make sure to read
https://www.prisma.io/docs/guides/database/developing-with-prisma-migrate/troubleshooting

[+] Added tables
  - booking
  - guest
  - payment
  - room
  - room_type

[*] Changed the `booking` table
  [+] Added foreign key on columns (guest_id)
  [+] Added foreign key on columns (room_id)

[*] Changed the `guest` table
  [+] Added unique index on columns (email)
  [+] Added unique index on columns (passport_details)

[*] Changed the `payment` table
  [+] Added foreign key on columns (booking_id)

[*] Changed the `room` table
  [+] Added unique index on columns (room_number)
  [+] Added foreign key on columns (type_id)

√ We need to reset the "public" schema at "localhost:5432"
Do you want to continue? All data will be lost. ... yes

Applying migration `20260319111656_final_migration`

The following migration(s) have been created and applied from new schema changes:

migrations/
  └─ 20260319111656_final_migration/
    └─ migration.sql

Your database is now in sync with your schema.

✔Generated Prisma Client (v5.22.0) to .\node_modules\@prisma\client in 105ms
```

Тестування оновленої бази даних за допомогою Prisma Client:

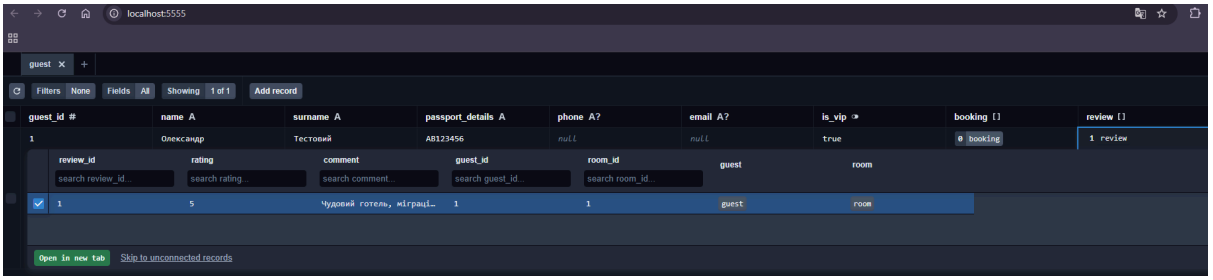
Щоб перевірити, чи правильно працюють нові поля та таблиці, було створено Node.js скрипт (test.js).

Скрипт успішно виконав наступні дії:
1.Створив нову кімнату.
2.Створив нового гостя із заповненим новим полем is_vip: true.
3.Створив запис у новій таблиці review, прив'язавши його до створеного гостя та кімнати.
4.Зробив запит до бази даних і вивів список відгуків разом із детальною інформацією про гостя (де видно статус is_vip) та кімнату.

```
C:\Users\user\Desktop\Lab 6>node test.js
=== Початок тестування нової бази даних ===
✅ Створено гостя VIP: Олександр Тестовий

✅ Додано новий відгук:
{
  review_id: 1,
  rating: 5,
  comment: 'Чудовий готель, міграції Prisma працюють ідеально!',
  guest_id: 1,
  room_id: 1
}

📋 Всі відгуки в базі даних:
[
  {
    review_id: 1,
    rating: 5,
    comment: 'Чудовий готель, міграції Prisma працюють ідеально!',
    guest_id: 1,
    room_id: 1,
    guest: {
      guest_id: 1,
      name: 'Олександр',
      surname: 'Тестовий',
      passport_details: 'AB123456',
      phone: null,
      email: null,
      is_vip: true
    },
    room: {
      room_id: 1,
      room_number: '101-VIP',
      status: 'вільний',
      type_id: null
    }
  }
]

=== Тестування завершено успішно ===
```

Перевірка даних через Prisma Studio:

Для зручного візуального перегляду бази даних та перевірки наявності записів було використано графічний інтерфейс Prisma Studio (команда npx prisma studio).



Висновок: Під час виконання лабораторної роботи я успішно засвоїв механізм міграцій Prisma ORM. Я навчився декларативно описувати зміни структури бази даних у файлі schema.prisma та безпечно переносити їх у СУБД PostgreSQL. Також я закріпив навички роботи з Prisma Client для виконання базових CRUD-операцій з урахуванням оновленої структури та нових зв'язків між таблицями.